

论文精读：

Gated Linear Attention Transformers

with Hardware-Efficient Training

Songlin Yang*, Bailin Wang*, Yikang Shen, Rameswar Panda, Yoon Kim

MIT & MIT-IBM Watson AI Lab

精读文档

March 6, 2026

Contents

1 摘要 (Abstract)

原文翻译

具有线性注意力的Transformer允许高效的并行训练，同时可以被表述为具有二维（矩阵值）隐状态的RNN，从而享有线性时间的推理复杂度。然而，线性注意力通常不如普通的softmax注意力。此外，线性注意力的当前实现缺乏I/O感知性，因此比高度优化的softmax注意力实现更慢。本工作描述了一种线性注意力的硬件高效算法，在内存移动与可并行性之间进行权衡。由此产生的实现，称为FlashLinearAttention，即使在短序列长度（如1K）上，作为独立层也比FlashAttention-2【[Dao 2023: FlashAttention-2, 加速softmax注意力的I/O感知算法; 本文以此为速度基准](#)】更快。然后我们将此算法推广到线性注意力的一个更具表达力的变体——具有数据依赖门控的线性注意力。当用作Transformer中标准注意力层的替代时，所得到的门控线性注意力（GLA）Transformer在中等规模的语言建模实验中，与LLaMA架构Transformer【[Touvron et al. 2023: LLaMA, Meta的开源基础语言模型; 本文的Transformer基线采用其架构](#)】以及最近的线性时间推理基线如RetNet【[Sun et al. 2023: RetNet, 使用全局衰减因子的线性注意力模型](#)】和Mamba【[Gu & Dao 2023: Mamba, 选择性状态空间模型; 本文的主要竞争对手](#)】表现出竞争力。GLA Transformer在长度泛化方面特别有效，使在2K上训练的模型能够泛化到超过20K的序列而无显著困惑度退化。在训练速度方面，GLA Transformer的吞吐量高于类似大小的Mamba模型。
GitHub: <https://github.com/sustcsonglin/flash-linear-attention>

通俗解释

这篇论文解决了两个核心问题：

问题1：线性注意力太慢。标准Transformer的softmax注意力计算量随序列长度平方增长，线性注意力虽然理论上更高效，但由于没有针对GPU硬件优化（比如没有充分利用GPU的快速片上缓存SRAM），实际运行反而比经过高度优化的FlashAttention-2更慢。

问题2：线性注意力效果不够好。线性注意力去掉了softmax中的非线性操作，导致模型表达能力下降。虽然RetNet等工作通过添加“衰减因子”来改善，但这个衰减因子是固定的（不随输入数据变化），不够灵活。

本文的解决方案：

1. 提出FlashLinearAttention——一种I/O感知的线性注意力硬件高效算法，比FlashAttention-2更快；
2. 提出GLA（Gated Linear Attention）——添加了**数据依赖**的门控机制（类似LSTM的遗忘门），让模型能根据输入内容灵活地“记住”或“遗忘”信息；
3. 在340M和1.3B参数规模上验证GLA的效果可与Transformer++和Mamba媲美。

打个比方：标准线性注意力像一个只会不停记录笔记却不会擦除的记事本，GLA则像一个智能记事本，能根据当前阅读内容自动决定哪些旧笔记该保留、哪些该擦掉。

关键概念/总结

1. **FlashLinearAttention**：线性注意力的I/O感知实现，利用分块（chunking）策略和SRAM缓存减少HBM访问。
2. **GLA**：带有细粒度、数据依赖门控的线性注意力变体，隐状态更新公式为 $\mathbf{S}_t = (\alpha_t^\top \mathbf{1}) \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$ 。
3. **训练效率**：GLA吞吐量显著高于Mamba（因为GLA能充分利用tensor core）。
4. **长度泛化**：GLA在2K训练后可泛化至20K+序列。

2 引言 (Introduction)

原文翻译

具有softmax注意力的Transformer【Vaswani et al. 2017: “Attention Is All You Need”，提出Transformer架构的开创性工作】享有高效的并行训练，但具有序列长度二次方的复杂度，从而激发了更多类RNN的模型以实现线性时间的序列建模。线性注意力用简单的点积替换指数相似函数，作用于（可能变换后的）键/查询向量，已成为经典softmax注意力的一个有前途的替代方案【Katharopoulos et al. 2020: 提出线性注意力的开创性工作，将Transformer与RNN统一】【Choromanski et al. 2021 (Performer): 使用随机特征近似softmax注意力】【Kasai et al. 2021: 将预训练Transformer微调为RNN】【Peng et al. 2021 (Random Feature Attention): 使用随机特征的线性注意力】。线性注意力的一个吸引人的特性是它允许一个“循环形式”，可以被表述为具有二维隐状态的线性RNN【Katharopoulos et al. 2020: 首次揭示线性注意力与RNN的等价关系】，从而实现线性时间推理。对于训练，线性注意力还允许一种亚二次方的“分块并行形式”，将序列分成不重叠的块，执行（串行的）块间循环计算，然后进行（并行的）块内计算【Hua et al. 2022 (GAU): 首次提出分块线性注意力形式】【Sun et al. 2023 (RetNet): 将分块形式推广到带衰减的线性注意力】【Lingle 2023 (VQ-Transformer): 使用向量量化的线性Transformer】，从而（部分地）保持并行训练。然而，现有的线性注意力算法不具有I/O感知性，因此在中等序列长度上实际运行比softmax注意力的优化实现更慢【Dao et al. 2022 (FlashAttention-1): 首个I/O感知的softmax注意力实现】【Dao 2023 (FlashAttention-2): 改进版FlashAttention，增加序列并行】。

从性能角度来看，线性注意力通常被发现不如普通softmax注意力，在语言建模中往往差距显著【Kasai et al. 2021: 实验显示线性注意力在语言建模上明显劣于softmax注意力】。最近的线性注意力变体如RetNet【Sun et al. 2023】和TransNormer-LLM【Qin et al. 2023b: TransNormerLLM，另一个带衰减的线性注意力大模型】通过在RNN更新前将当前隐状态乘以衰减因子获得了显著改进。然而，这些工作使用全局的、数据无关的衰减因子，尽管在一维RNN中，数据依赖的门控机制已被证明对性能至关重要【van der Westhuizen & Lasenby 2018: “The Unreasonable Effectiveness of the Forget Gate”，证明遗忘门的关键作用】【Qin et al. 2023c (HGRN): 分层门控循环网络，使用门控机制】。而且即使有了衰减因子，线性注意力Transformer在从头预训练时仍然不如最强的Transformer架构。

本工作开发了一种线性注意力的硬件高效算法，并将其应用于训练一种与softmax注意力竞争的线性注意力变体。我们首先讨论在现代GPU上优化普通线性注意力的各个方面，并基于这些原则给出两种I/O感知算法（针对不同训练设置），称为FlashLinearAttention，它比FlashAttention-2甚至在短（如1K）序列上更快。然后我们描述一个具有数据依赖门控机制的线性注意力层，并展示FlashLinearAttention如何推广到门控情况。我们在中等规模的语言建模基准上研究所得到的门控线性注意力（GLA）Transformer，训练340M/1.3B参数的模型，分别在15B/100B个token上训练。我们发现GLA Transformer在强LLaMA架构Transformer基线【Touvron et al. 2023】以及最近的线性时间序列模型如RetNet【Sun et al. 2023】和Mamba【Gu & Dao 2023】面前表现良好。GLA Transformer在线性循环模型中在长度泛化和需要记忆检索的任务上特别强大。在训练速度方面，GLA Transformer的吞吐量显著高于类似大小的Mamba模型。

通俗解释

引言交代了三个层面的问题和对应的贡献：

层面1——速度问题：线性注意力在“纸面上”复杂度低，但实际跑起来并不快。为什么？因为现有实现没有考虑GPU的内存层级（SRAM vs HBM），大量时间浪费在数据

搬运上。类比：你有一张超大的Excel表格，虽然计算简单，但每次都要从硬盘读取而不是从内存中读取，所以反而比一个计算复杂但全在内存中操作的程序更慢。

层面2——效果问题：线性注意力丢掉了softmax的非线性，表达力弱了。RetNet通过“全局衰减”（类似每个时间步乘以一个固定的0.99）来给近期信息更高权重，但这个衰减对所有输入都一样。而GLA使用“数据依赖的门控”——每个时间步根据当前输入决定遗忘多少，就像LSTM的遗忘门一样灵活。

层面3——可扩展性问题：Mamba虽然也用了数据依赖的门控，但它的实现方式不能利用GPU的tensor core（专门加速矩阵乘法的硬件单元），导致隐状态维度受限、训练效率低。GLA通过巧妙的“分块”策略，让大部分计算可以用tensor core加速。

注意事项

线性注意力 $\neq$ 线性复杂度的注意力。“线性注意力”特指用线性核替换softmax的注意力变体，其“并行形式”仍然是 $O(L^2d)$ 的。只有使用“循环形式”或“分块形式”才能实现低于二次方的复杂度。不要混淆“线性注意力”和“线性复杂度”。

3 背景：线性注意力 (Background: Linear Attention)

原文翻译

我们首先简要介绍线性注意力层的背景。在符号上，我们使用粗体大写字母表示矩阵（如 $\mathbf{S}$ 、 $\mathbf{Q}$ ），粗体小写字母表示向量（如 $\mathbf{q}_t$ 、 $\mathbf{k}_t$ ），斜体大写表示可学习参数矩阵（如 $\mathbf{W}_K$ ）。我们通常使用相同的字母表示矩阵的行，例如 $\mathbf{q}_t$ 是 $\mathbf{Q}$ 的第 t 行。

3.1 并行形式与循环形式

原文翻译

标准自回归Transformer采用softmax注意力机制，给定输入序列 $\mathbf{X} \in \mathbb{R}^{L \times d}$ （这里 L 是长度， d 是隐藏维度），通过以下方式计算输出 $\mathbf{O} \in \mathbb{R}^{L \times d}$ ：

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}\mathbf{W}_Q, \mathbf{X}\mathbf{W}_K, \mathbf{X}\mathbf{W}_V, \quad (1)$$

$$\mathbf{O} = \text{softmax}((\mathbf{Q}\mathbf{K}^\top) \odot \mathbf{M})\mathbf{V}, \quad (2)$$

其中 $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$ 是可学习矩阵， $\mathbf{M} \in \{-\infty, 1\}^{L \times L}$ 是防止模型关注未来tokens的掩码，即当 $i \geq j$ 时 $\mathbf{M}_{ij} = 1$ ，当 $i < j$ 时 $\mathbf{M}_{ij} = -\infty$ 。（这里为简单起见假设单个注意力头。）以上注意力的并行形式可以在给定完整输入 $\mathbf{X}$ 的情况下并行计算 $\mathbf{O}$ ，从而实现高效训练。然而，在推理时Transformer必须使用以下循环形式：

$$\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t = \mathbf{x}_t\mathbf{W}_Q, \mathbf{x}_t\mathbf{W}_K, \mathbf{x}_t\mathbf{W}_V, \quad (3)$$

$$\mathbf{o}_t = \frac{\sum_{i=1}^t \exp(\mathbf{q}_t\mathbf{k}_i^\top) \mathbf{v}_i}{\sum_{i=1}^t \exp(\mathbf{q}_t\mathbf{k}_i^\top)}, \quad (4)$$

它根据当前token的表示 $\mathbf{x}_t \in \mathbb{R}^{1 \times d}$ 计算查询($\mathbf{q}_t$)、键($\mathbf{k}_t$)和值($\mathbf{v}_t$)向量，并在（不断增长的）键集 $\{\mathbf{k}_1, \dots, \mathbf{k}_t\}$ 和值集 $\{\mathbf{v}_1, \dots, \mathbf{v}_t\}$ （即“KV缓存”）上执行注意力。

线性注意力机制【Katharopoulos et al. 2020: 线性注意力的原始论文】用核 $k(\mathbf{x}, \mathbf{y})$ 及其对应的特征映射 ϕ （即 $k(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$ ）替换 $\exp(\mathbf{q}_t\mathbf{k}_i^\top)$ 。这简化了 $\mathbf{o}_t$ 的计算，因为：

$$\mathbf{o}_t = \frac{\sum_{i=1}^t \phi(\mathbf{q}_t)\phi(\mathbf{k}_i)^\top \mathbf{v}_i}{\sum_{i=1}^t \phi(\mathbf{q}_t)\phi(\mathbf{k}_i)^\top} = \frac{\phi(\mathbf{q}_t) \sum_{i=1}^t \phi(\mathbf{k}_i)^\top \mathbf{v}_i}{\phi(\mathbf{q}_t) \sum_{i=1}^t \phi(\mathbf{k}_i)^\top}. \quad (5)$$

令 $\mathbf{S}_t = \sum_{i=1}^t \phi(\mathbf{k}_i)^\top \mathbf{v}_i$, $\mathbf{z}_t = \sum_{i=1}^t \phi(\mathbf{k}_i)^\top$, 其中 $\mathbf{S}_t \in \mathbb{R}^{d \times d}$, $\mathbf{z}_t \in \mathbb{R}^{d \times 1}$, 我们可以将上式改写为一个RNN:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \phi(\mathbf{k}_t)^\top \mathbf{v}_t, \quad \mathbf{z}_t = \mathbf{z}_{t-1} + \phi(\mathbf{k}_t)^\top, \quad \mathbf{o}_t = \frac{\phi(\mathbf{q}_t) \mathbf{S}_t}{\phi(\mathbf{q}_t) \mathbf{z}_t}. \quad (6)$$

尽管已经探索了各种核【Kasai et al. 2021】【Peng et al. 2021】，但最近的工作发现线性核（即将 ϕ 设为恒等映射）不带归一化器在实践中效果良好【Sun et al. 2023】。这产生了如下（非归一化的）线性注意力层更新方程：

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \quad \mathbf{o}_t = \mathbf{q}_t \mathbf{S}_t. \quad (7)$$

公式 7清楚地表明，线性注意力层本质上是一个具有矩阵值隐状态 $\mathbf{S}_t$ 的线性循环层，通过外积 $\mathbf{k}_t^\top \mathbf{v}_t = (\mathbf{x}_t W_K)^\top (\mathbf{x}_t W_V)$ 来更新。^a 因果线性注意力的并行形式，其复杂度仍然是 L 的二次方，由 $\mathbf{O} = ((\mathbf{QK}^\top) \odot \mathbf{M}) \mathbf{V}$ 给出，其中 $\mathbf{M} \in \{0, 1\}^{L \times L}$ 是一个掩码， $\mathbf{M}_{ij} = 1$ 当 $i \geq j$ ， $\mathbf{M}_{ij} = 0$ 当 $i < j$ 。由于 $\mathbf{M}$ 的存在，无法利用矩阵乘法的结合律将并行形式复杂度从二次降到线性。^b

^a这种隐状态随时间变化的矩阵值模型也被称为“快速权重（fast weights）”，其与Transformer的联系在最近的工作中有所探讨。

^b不带 $\mathbf{M}$ 时，可以将 $(\mathbf{QK}^\top) \mathbf{V}$ 变换为 $\mathbf{Q}(\mathbf{K}^\top \mathbf{V})$ ，将复杂度从 $O(L^2 d)$ 降低到 $O(Ld^2)$ 。

核心公式推导：从Softmax注意力到线性注意力

步骤1：理解softmax注意力

对于第 t 个位置的输出：

$$\mathbf{o}_t = \frac{\sum_{i=1}^t \exp(\mathbf{q}_t \mathbf{k}_i^\top) \mathbf{v}_i}{\sum_{i=1}^t \exp(\mathbf{q}_t \mathbf{k}_i^\top)}$$

这里 $\mathbf{q}_t \in \mathbb{R}^{1 \times d}$, $\mathbf{k}_i \in \mathbb{R}^{1 \times d}$, $\mathbf{v}_i \in \mathbb{R}^{d \times 1}$ 。 $\mathbf{q}_t \mathbf{k}_i^\top$ 是标量（ $1 \times d$ 乘 $d \times 1$ ）。

步骤2：线性注意力的关键替换

将 $\exp(\mathbf{q}_t \mathbf{k}_i^\top)$ 替换为 $\phi(\mathbf{q}_t) \phi(\mathbf{k}_i)^\top$ 。关键观察——因为不再有softmax的“max”技巧，分子可以提取公因子：

$$\mathbf{o}_t = \frac{\phi(\mathbf{q}_t) \sum_{i=1}^t \phi(\mathbf{k}_i)^\top \mathbf{v}_i}{\phi(\mathbf{q}_t) \sum_{i=1}^t \phi(\mathbf{k}_i)^\top}$$

步骤3：定义隐状态 $\mathbf{S}_t$

令 $\mathbf{S}_t = \sum_{i=1}^t \phi(\mathbf{k}_i)^\top \mathbf{v}_i$, 维度分析：

- $\phi(\mathbf{k}_i)^\top$: $(d \times 1)$
- $\mathbf{v}_i$: $(1 \times d)$
- $\phi(\mathbf{k}_i)^\top \mathbf{v}_i$: $(d \times d)$ (外积)
- 因此 $\mathbf{S}_t \in \mathbb{R}^{d \times d}$

这就是“矩阵值隐状态”——相比传统RNN的向量隐状态 $\mathbb{R}^d$ ，这里的隐状态是 $\mathbb{R}^{d \times d}$ 的矩阵，信息容量大大增加。

步骤4：递推关系

由于 $\mathbf{S}_t = \sum_{i=1}^t \phi(\mathbf{k}_i)^\top \mathbf{v}_i = \sum_{i=1}^{t-1} \phi(\mathbf{k}_i)^\top \mathbf{v}_i + \phi(\mathbf{k}_t)^\top \mathbf{v}_t = \mathbf{S}_{t-1} + \phi(\mathbf{k}_t)^\top \mathbf{v}_t$

这就是一个线性RNN！当 $\phi = \text{identity}$ 时， $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$ 。

数值例子（ $d = 2$ ）：

设 $\mathbf{k}_1 = [1, 2]$, $\mathbf{v}_1 = [3, 4]$, 则：

$$\mathbf{k}_1^\top \mathbf{v}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix}$$

设 $\mathbf{k}_2 = [0.5, 1]$, $\mathbf{v}_2 = [2, 1]$, 则 $\mathbf{S}_2 = \mathbf{S}_1 + \mathbf{k}_2^\top \mathbf{v}_2 = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix} + \begin{bmatrix} 1 & 0.5 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 4.5 \\ 8 & 9 \end{bmatrix}$
 查询 $\mathbf{q}_2 = [1, 0.5]$ 的输出: $\mathbf{o}_2 = \mathbf{q}_2 \mathbf{S}_2 = [1, 0.5] \begin{bmatrix} 4 & 4.5 \\ 8 & 9 \end{bmatrix} = [8, 9]$

3.2 分块并行形式 (Chunkwise Parallel Form)

原文翻译

线性注意力的分块并行形式在并行形式和循环形式之间取得平衡 **【Hua et al. 2022 (GAU): 首次提出分块并行形式】** **【Sun et al. 2023】**, 允许亚二次方的、部分并行训练。形式上, 假设输入 $\mathbf{X}$ 现在被分成不重叠的块, 每个块长度为 C 。令 $\mathbf{S}_{[i]} \in \mathbb{R}^{d \times d}$ 为处理 i 个块后的块级隐状态, 即 $\mathbf{S}_{[i]} := \mathbf{S}_{iC}$ 。进一步令 $\mathbf{Q}_{[i]} := \mathbf{Q}_{iC+1:(i+1)C+1} \in \mathbb{R}^{C \times d}$ 为第 i 块对应的查询向量; $\mathbf{K}_{[i]}$ 、 $\mathbf{V}_{[i]}$ 、 $\mathbf{O}_{[i]}$ 类似定义。块间递推 (对 $i \in [0, 1, \dots, \frac{L}{C} - 1]$) 为:

$$\mathbf{S}_{[i+1]} = \mathbf{S}_{[i]} + \underbrace{\sum_{j=iC+1}^{(i+1)C} \mathbf{k}_j^\top \mathbf{v}_j}_{\mathbf{K}_{[i]}^\top \mathbf{V}_{[i]}} \in \mathbb{R}^{d \times d}. \quad (8)$$

块内并行计算输出:

$$\mathbf{O}_{[i+1]} = \underbrace{\mathbf{Q}_{[i+1]} \mathbf{S}_{[i]}}_{\text{块间: } \mathbf{O}_{[i+1]}^{\text{inter}}} + \underbrace{((\mathbf{Q}_{[i+1]} \mathbf{K}_{[i+1]}^\top) \odot \mathbf{M}) \mathbf{V}_{[i+1]}}_{\text{块内: } \mathbf{O}_{[i+1]}^{\text{intra}}}. \quad (9)$$

训练复杂度为 $O(\frac{L}{C}(C^2d + Cd^2)) = O(LCd + Ld^2)$, 当 $L > d$ 时小于 $O(L^2d)$ 。设 $C = L$ 恢复并行形式, $C = 1$ 恢复循环形式。

通俗解释

分块并行是本文最核心的算法思想。想象你有一本1000页的书(序列), 你把它分成每10页一组(块大小 $C = 10$), 共100组。

块间 (inter-chunk) 操作是串行的: 你必须按顺序处理每一组, 因为第2组需要第1组的“总结”。这个“总结”就是隐状态 $\mathbf{S}_{[i]}$, 它是一个 $d \times d$ 的矩阵, 记录了到目前为止所有key-value的累积信息。

块内 (intra-chunk) 操作是并行的: 在一个10页的组内部, 可以像标准注意力那样并行计算每一页与组内其他页的关系。

块大小 C 的权衡:

- C 大: 块内计算多 ($O(C^2d)$), 并行度高, 但接近全并行形式的二次复杂度
- C 小: 块间计算多 ($O(Ld^2)$), 串行度高, 但总FLOPs少
- 最优 C 在 $\sqrt{d}$ 附近, 此时总复杂度 $O(Ld\sqrt{d})$

分块形式的复杂度分析

序列被分成 $N = L/C$ 块。

块间递推: $\mathbf{S}_{[i+1]} = \mathbf{S}_{[i]} + \mathbf{K}_{[i]}^\top \mathbf{V}_{[i]}$

- $\mathbf{K}_{[i]}^\top \mathbf{V}_{[i]}$: $(d \times C) \times (C \times d) = O(Cd^2)$

· 共 N 块: $O(NCd^2) = O(Ld^2)$

块内并行: $\mathbf{O}_{[i+1]}^{\text{intra}} = ((\mathbf{Q}_{[i+1]} \mathbf{K}_{[i+1]}^T) \odot \mathbf{M}) \mathbf{V}_{[i+1]}$

· $\mathbf{Q}_{[i+1]} \mathbf{K}_{[i+1]}^T$: $(C \times d) \times (d \times C) = O(C^2 d)$

· 乘 $\mathbf{V}_{[i+1]}$: $(C \times C) \times (C \times d) = O(C^2 d)$

· 共 N 块: $O(NC^2 d) = O(LC d)$

块间传播: $\mathbf{O}_{[i+1]}^{\text{inter}} = \mathbf{Q}_{[i+1]} \mathbf{S}_{[i]}$: $(C \times d) \times (d \times d) = O(Cd^2)$, 共 $O(Ld^2)$ 。

总复杂度: $O(LC d + Ld^2)$ 。当 $C \ll L$ 时远小于 $O(L^2 d)$ 。

4 硬件高效的线性注意力 (Hardware-Efficient Linear Attention)

原文翻译

我们描述FlashLinearAttention——一种I/O感知的、硬件高效的线性注意力算法，精神上类似于FlashAttention **【Dao et al. 2022: FlashAttention-1】** **【Dao 2023: FlashAttention-2】**。我们首先讨论在实际高效实现中应考虑硬件方面。

4.1 硬件高效算法原则

原文翻译

一个高效的算法应该了解现代硬件上的计算模型、内存层级和专用计算单元。

占用率 (Occupancy)。GPU有许多并行执行的线程；线程被分组为线程块，在流式多处理器 (SM) 上执行。为了保持高GPU占用率（即正在使用的GPU资源比例），需要使用足够数量的SM。在批量大小趋向较小的大规模训练和长序列建模场景中，在时间维度上并行化可以实现高GPU占用率 **【Dao 2023】**。

专用计算单元。用于神经网络训练的现代硬件通常具有专用计算单元（如NVIDIA GPU上的**tensor core**、TPU上的矩阵乘法单元），可以显著加速矩阵乘法；例如，在A100上半精度矩阵乘法在**tensor core**上可以比在CUDA core上快大约16倍。

内存层级。GPU具有较大但较慢的全局GPU内存（高带宽内存；HBM）和较小但较快的共享内存（SRAM）。对SRAM的优化利用以减少HBM I/O成本因此可以带来显著的加速。

通俗解释

这里介绍了GPU编程的三个核心概念：

1. 占用率：GPU有很多“工人”（SM），如果只有少数工人在干活，其他都在闲着，那就浪费了。当batch size小时（如长序列训练），需要在序列维度上找更多并行性来让所有工人都有事做。

2. Tensor Core：这是GPU里专门做矩阵乘法的“超级加速器”，能把矩阵乘法加速16倍！但它有条件——只能做特定格式的矩阵乘法（半精度、维度是16的倍数）。所以算法设计的关键是尽量把计算转化为矩阵乘法。Mamba不能用**tensor core**，这是它的一个弱点。

3. 内存层级：HBM（约40-80GB，带宽2TB/s）和SRAM（约20MB，带宽19TB/s以上）。SRAM快10倍但小2000倍。FlashAttention的核心思想就是尽量在SRAM上完成计算，减少HBM读写。

注意事项

“快”不等于“FLOPs少”。Mamba的总FLOPs可能比GLA少，但因为Mamba不能利用tensor core（16倍加速!），实际wall-clock时间反而更长。这就是为什么本文强调“hardware-efficient”而不仅仅是“computation-efficient”。

4.2 线性注意力的硬件考量

原文翻译

循环形式。循环形式的基本实现将所有时间步的2D隐状态存储在HBM中，导致高I/O成本。可以通过在反向传播时重计算隐状态来减少I/O成本，但循环更新中的逐元素操作不能利用tensor core，导致低算术强度。虽然理论上可以通过并行扫描算法并行化线性递推，但这需要实体化每个时间步的2D隐状态，内存I/O负担大。

并行形式。并行形式可以像FlashAttention那样高效，但由于二次复杂度的高FLOPs，长序列训练仍然昂贵。

分块形式。分块并行形式在并行和循环形式之间插值，并有额外的“参数” C ，使得更容易进行上述权衡的细粒度优化。与循环形式不同，大多数操作可以通过矩阵乘法完成，从而能够使用tensor core（如果 C 设为16的倍数）。尽管分块训练算法在文献中已有讨论，但大多数实现不具有I/O感知性，因此对于中等序列长度比FlashAttention慢。

关键概念/总结

三种形式的权衡：

特性	循环形式	并行形式	分块形式
FLOPs	$O(Ld^2)$ 最低	$O(L^2d)$ 最高	$O(LCd + Ld^2)$ 中间
可并行性	无	完全并行	部分并行
Tensor Core	不能用	能用	能用
I/O成本	高（元素操作）	可优化	可优化

分块形式综合了两者的优点，是本文的核心算法基础。

4.3 FlashLinearAttention 算法

原文翻译

我们描述分块形式的I/O感知、硬件高效实现。我们给出两个版本，其前向和反向传播取决于是否将块级隐状态 $\mathbf{S}_{[n]}$ 实体化到HBM。在高层次上，我们使用分块（tiling）技术来逐块加载张量，并在片上重复使用张量块以尽可能减少多次HBM I/O。例如，当 $\mathbf{Q}_{[n]}$ 被加载到SRAM时， $\mathbf{Q}_{[n]}\mathbf{S}$ 和 $(\mathbf{Q}_{[n]}\mathbf{K}_{[n]}^\top \odot \mathbf{M})\mathbf{V}_{[n]}$ 都可以在片上计算，避免加载 $\mathbf{Q}_{[n]}$ 两次，从而节省HBM I/O。

非实体化版本对 $n \in [N]$ 顺序计算 $\mathbf{O}_{[n]}$ ，使用SRAM临时存储 $\mathbf{S}_{[n]}$ ，内存高效。此版本在batch size、头数和头维度上并行，但缺乏序列级并行。当batch size大时，这种并行度足以实现高GPU占用率。

实体化版本首先执行块间递推并将所有 $\mathbf{S}_{[n]}$ 存储到HBM。然后 $\mathbf{O}_{[n]}$ 可以对所有块并行计算。这种方法提供更好的并行性但增加约10–20%的内存占用。我们通过重计算来缓解——前向传播后丢弃隐状态，在反向传播时重新计算。

下面是FlashLinearAttention前向传播的伪代码（算法 1）。

Algorithm 1 FlashLinearAttention: 前向传播

输入: $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{L \times d}$, 块大小 $C \in [L]$, $\text{materialize} \in \{\text{True}, \text{False}\}$

将 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 分成 $N = \frac{L}{C}$ 块 $\{\mathbf{Q}_{[1]} \dots \mathbf{Q}_{[N]}\}, \{\mathbf{K}_{[1]} \dots \mathbf{K}_{[N]}\}, \{\mathbf{V}_{[1]} \dots \mathbf{V}_{[N]}\}$, 每块大小 $C \times d$ 。

初始化 $\mathbf{S} = \mathbf{0} \in \mathbb{R}^{d \times d}$ 于 SRAM 上

在片上构造因果掩码 $\mathbf{M} \in \mathbb{R}^{C \times C}$

if materialize then

▷ 实体化版本

 for $n \leftarrow 1, N$ do

 将 $\mathbf{S}$ 存储到 HBM 为 $\mathbf{S}_{[n]}$

 从 HBM 加载 $\mathbf{K}_{[n]}, \mathbf{V}_{[n]} \in \mathbb{R}^{C \times d}$ 到 SRAM

 在片上计算 $\mathbf{S} = \mathbf{S} + \mathbf{K}_{[n]}^\top \mathbf{V}_{[n]}$

 end for

 parfor $n \leftarrow 1, N$ do

 从 HBM 加载 $\mathbf{Q}_{[n]}, \mathbf{K}_{[n]}, \mathbf{V}_{[n]}, \mathbf{S}_{[n]}$ 到 SRAM

 在片上计算 $\mathbf{O}' = \mathbf{Q}_{[n]} \mathbf{S}_{[n]} + (\mathbf{Q}_{[n]} \mathbf{K}_{[n]}^\top \odot \mathbf{M}) \mathbf{V}_{[n]}$

 将 $\mathbf{O}'$ 存储到 HBM 为 $\mathbf{O}_{[n]}$

 end parfor

 return $\mathbf{O} = \{\mathbf{O}_{[1]} \dots \mathbf{O}_{[N]}\}, \mathbf{S} = \{\mathbf{S}_{[1]} \dots \mathbf{S}_{[N]}\}$

else

▷ 非实体化版本

 for $n \leftarrow 1, N$ do

 从 HBM 加载 $\mathbf{Q}_{[n]}, \mathbf{K}_{[n]}, \mathbf{V}_{[n]} \in \mathbb{R}^{C \times d}$ 到 SRAM

 在片上计算 $\mathbf{O}' = \mathbf{Q}_{[n]} \mathbf{S} + (\mathbf{Q}_{[n]} \mathbf{K}_{[n]}^\top \odot \mathbf{M}) \mathbf{V}_{[n]}$

 在片上计算 $\mathbf{S} = \mathbf{S} + \mathbf{K}_{[n]}^\top \mathbf{V}_{[n]}$

 将 $\mathbf{O}'$ 存储到 HBM 为 $\mathbf{O}_{[n]}$

 end for

 return $\mathbf{O} = \{\mathbf{O}_{[1]} \dots \mathbf{O}_{[N]}\}$

end if

算法 1 逐行解读

实体化版本分两个阶段:

阶段1 (串行, 第4-8行):

- 第5行: 把当前 $\mathbf{S}$ 写到HBM保存——因为后面并行阶段需要用。
- 第6行: 从HBM读 $\mathbf{K}_{[n]}, \mathbf{V}_{[n]}$ 到快速SRAM。
- 第7行: 在SRAM上做矩阵乘法更新隐状态: $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{K}_{[n]}^\top \mathbf{V}_{[n]}$, 这是 $O(Cd^2)$ 的操作, 可用tensor core。

阶段2 (并行, 第9-13行):

- 所有 N 个块可以**同时**在不同SM上计算! 因为每个块只需要自己的 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 和已保存的 $\mathbf{S}_{[n]}$ 。
- 第11行的两项分别是**块间贡献** ($\mathbf{Q}_{[n]} \mathbf{S}_{[n]}$, 来自之前所有块的信息) 和**块内贡献** (标准线性注意力, 只看当前块内部)。

非实体化版本把两个阶段合成一个串行循环: 不保存 $\mathbf{S}$ 到HBM, 而是在SRAM上维护 $\mathbf{S}$, 边算输出边更新 $\mathbf{S}$ 。省内存但无法在序列维度并行。

5 门控线性注意力 (Gated Linear Attention)

原文翻译

公式 7 中的线性递推没有衰减项或遗忘门，而这在 RNN 中已被证明至关重要【Hochreiter & Schmidhuber 1997 (LSTM): 提出 LSTM, 包含遗忘门】【Cho et al. 2014 (GRU): 门控循环单元】【van der Westhuizen & Lasenby 2018: 证明遗忘门的重要性】。缺乏衰减项使模型难以“遗忘”信息，并被假设为线性注意力在长上下文任务中不稳定的部分原因【Buckman & Gelada 2024: 分析线性 Transformer 的稳定性问题】。最近的工作【Sun et al. 2023 (RetNet)】【Qin et al. 2023b (TransNormerLLM)】通过将全局的、非数据依赖的衰减因子 $\gamma \in (0, 1)$ 纳入线性注意力获得更好的性能： $\mathbf{S}_t = \gamma \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$ 。使用单个 γ 旨在保留注意力风格的并行形式以实现高效训练。本工作考虑线性注意力的数据依赖门控机制。我们展示尽管具有更具表达力的门控因子，所得到的门控线性注意力 (GLA) 层仍然允许硬件高效的分块形式进行高效训练。

5.1 GLA 的循环形式与并行形式

原文翻译

循环形式。 GLA 有一个二维遗忘门 $\mathbf{G}_t \in (0, 1)^{d_k \times d_v}$ 随时间变化：

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \quad (10)$$

其中我们现在允许隐状态具有不同的维度。这种基于 Hadamard 积的循环形式非常通用，涵盖了许多最近的具有 2D 隐状态的 RNN。

下面的表 1 列出了各种模型对 $\mathbf{G}_t$ 的参数化方式。

Table 1: 各种模型的门控线性注意力形式，它们在 $\mathbf{G}_t$ 的参数化方式上有所不同。

模型	参数化	可学习参数
Mamba	$\mathbf{G}_t = \exp(-(\mathbf{1}^\top \alpha_t)) \odot \exp(\mathbf{A})$	$\mathbf{A} \in \mathbb{R}^{d_k \times d_v}$
Mamba-2	$\mathbf{G}_t = \gamma_t \mathbf{1}^\top \mathbf{1}$, 标量门	$W_\gamma \in \mathbb{R}^{d \times 1}$
mLSTM	$\mathbf{G}_t = \gamma_t \mathbf{1}^\top \mathbf{1}$, $\gamma_t = \sigma(\mathbf{x}_t W_\gamma)$	$W_\gamma \in \mathbb{R}^{d \times 1}$
DFW	$\mathbf{G}_t = \alpha_t^\top \beta_t$, 全低秩	$W_\alpha \in \mathbb{R}^{d \times d_k}, W_\beta \in \mathbb{R}^{d \times d_v}$
GateLoop	$\mathbf{G}_t = \alpha_t^\top \mathbf{1}$, 复数门	$W_{\alpha_1}, W_{\alpha_2} \in \mathbb{R}^{d \times d_k}$
HGRN-2	$\mathbf{G}_t = \alpha_t^\top \mathbf{1}$, 可学习下界	$W_\alpha \in \mathbb{R}^{d \times d_k}$
RWKV-6	$\mathbf{G}_t = \alpha_t^\top \mathbf{1}$, 双重指数	$W_\alpha \in \mathbb{R}^{d \times d_k}$
GLA	$\mathbf{G}_t = \alpha_t^\top \mathbf{1}$, 低秩+温度	$W_{\alpha_1} \in \mathbb{R}^{d \times 16}, W_{\alpha_2} \in \mathbb{R}^{16 \times d_k}$

表 1 解读

这张表展示了不同模型如何设计“遗忘门” $\mathbf{G}_t$ 。核心区别在于：

粗粒度 vs 细粒度： Mamba-2/mLSTM 使用标量门 γ_t ，即所有维度共享同一个遗忘率。GLA/GateLoop/HGRN-2 使用向量门 α_t ，每个 key 维度有自己的遗忘率。Mamba/DFW 使用矩阵门（完全低秩），最灵活但也最贵。

GLA 的设计选择： 使用 $\mathbf{G}_t = \alpha_t^\top \mathbf{1}$ ，即 α_t 是 d_k 维向量， $\mathbf{1}$ 表示 value 维度全部共享。这是标量和矩阵之间的折中——足够灵活，且允许高效的分块并行形式。

GLA 的参数效率： α_t 通过低秩线性层 ($d \rightarrow 16 \rightarrow d_k$) 计算，只需 $16(d + d_k)$ 个参数。

原文翻译

本文采用标量和完全低秩参数化之间的折中，使用 $\mathbf{G}_t = \boldsymbol{\alpha}_t^\top \mathbf{1}$ 。这产生以下循环形式：

$$\mathbf{S}_t = (\boldsymbol{\alpha}_t^\top \mathbf{1}) \odot \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t = \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t, \quad (11)$$

其中 $\boldsymbol{\alpha}_t$ 通过低秩线性层后接sigmoid在 $\mathbf{x}_t$ 上参数化。

并行形式。 展开公式 11 得到：

$$\mathbf{S}_t = \sum_{i=1}^t \left(\left(\prod_{j=i+1}^t \boldsymbol{\alpha}_j^\top \mathbf{1} \right) \odot \mathbf{k}_i^\top \mathbf{v}_i \right)$$

令 $\mathbf{b}_t := \prod_{j=1}^t \boldsymbol{\alpha}_j$ ，可改写为：

$$\mathbf{o}_t = \mathbf{q}_t \mathbf{S}_t = \sum_{i=1}^t (\mathbf{q}_t \odot \mathbf{b}_t) \left(\frac{\mathbf{k}_i}{\mathbf{b}_i} \right)^\top \mathbf{v}_i$$

矩阵形式为：

$$\mathbf{O} = \left(\left(\underbrace{(\mathbf{Q} \odot \mathbf{B}) \left(\frac{\mathbf{K}}{\mathbf{B}} \right)^\top}_{\mathbf{P}} \right) \odot \mathbf{M} \right) \mathbf{V}.$$

然而此形式数值不稳定，因为 $\mathbf{b}_t$ 是 $(0, 1)$ 中值的累积乘积，当 t 大时极小。因此在对数空间计算 $\mathbf{P}$ ：

$$\mathbf{P}_{ij} = \sum_{k=1}^d \mathbf{Q}_{ik} \mathbf{K}_{jk} \exp(\log \mathbf{B}_{ik} - \log \mathbf{B}_{jk}), \quad i \geq j. \quad (12)$$

但此式不能表示为标准矩阵乘法，不能利用半精度tensor core。

GLA并行形式推导详解

起点： $\mathbf{S}_t = \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t$

展开递推：

$$\begin{aligned} \mathbf{S}_t &= \text{Diag}(\boldsymbol{\alpha}_t) [\text{Diag}(\boldsymbol{\alpha}_{t-1}) \mathbf{S}_{t-2} + \mathbf{k}_{t-1}^\top \mathbf{v}_{t-1}] + \mathbf{k}_t^\top \mathbf{v}_t \\ &= \text{Diag}(\boldsymbol{\alpha}_t) \text{Diag}(\boldsymbol{\alpha}_{t-1}) \mathbf{S}_{t-2} + \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{k}_{t-1}^\top \mathbf{v}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t \end{aligned}$$

完全展开后： $\mathbf{S}_t = \sum_{i=1}^t \prod_{j=i+1}^t \text{Diag}(\boldsymbol{\alpha}_j) \cdot \mathbf{k}_i^\top \mathbf{v}_i$ 。

由于 $\prod_{j=i+1}^t \text{Diag}(\boldsymbol{\alpha}_j) = \text{Diag}(\prod_{j=i+1}^t \boldsymbol{\alpha}_j)$ （对角矩阵连乘=元素乘积的对角矩阵），且对角矩阵与外积的乘法等价于Hadamard积，得到：

$$\mathbf{S}_t = \sum_{i=1}^t \left(\left(\frac{\mathbf{b}_t}{\mathbf{b}_i} \right)^\top \mathbf{1} \right) \odot \mathbf{k}_i^\top \mathbf{v}_i$$

利用混合积性质： $(\mathbf{a}^\top \mathbf{1}) \odot (\mathbf{c}^\top \mathbf{d}) = (\mathbf{a} \odot \mathbf{c})^\top \mathbf{d}$

所以 $\mathbf{o}_t = \mathbf{q}_t \mathbf{S}_t = \sum_{i=1}^t (\mathbf{q}_t \odot \mathbf{b}_t) (\mathbf{k}_i / \mathbf{b}_i)^\top \mathbf{v}_i$

这里 $\mathbf{q}_t \odot \mathbf{b}_t$ 相当于“衰减调整后的查询”， $\mathbf{k}_i / \mathbf{b}_i$ 相当于“衰减调整后的键”。

数值稳定性问题： $\mathbf{b}_t = \prod_{j=1}^t \boldsymbol{\alpha}_j$ ，每个 $\boldsymbol{\alpha}_j \in (0, 1)$ 。如果 $t = 1000$ 且平均 $\alpha \approx 0.99$ ，则 $\mathbf{b} \approx 0.99^{1000} \approx 4.3 \times 10^{-5}$ ，非常小！ $\mathbf{k}_i / \mathbf{b}_i$ 会爆炸。

解决方案： 在对数空间计算： $\log(b_t/b_i) = \log b_t - \log b_i = \sum_{j=i+1}^t \log \alpha_j$ ，这是稳定的差值。

5.2 GLA的分块并行形式

原文翻译

我们推导出类似于基本线性注意力分块形式的GLA分块形式。对于块间递推，我们定义：

$$\boxtimes_{iC+j} = \frac{b_{iC+j}}{b_{iC}}, \quad \boxtimes_{iC+j} = \frac{b_{(i+1)C}}{b_{iC+j}}, \quad \gamma_{i+1} = \frac{b_{(i+1)C}}{b_{iC}},$$

$$\mathbf{S}_{[i+1]} = (\gamma_{i+1}^\top \mathbf{1}) \odot \mathbf{S}_{[i]} + (\mathbf{K}_{[i+1]} \odot \boxtimes_{[i+1]})^\top \mathbf{V}_{[i+1]},$$

$$\mathbf{O}_{[i+1]}^{\text{inter}} = (\mathbf{Q}_{[i+1]} \odot \boxtimes_{[i+1]}) \mathbf{S}_{[i]}.$$

直觉上， $\boxtimes_{[i+1]}$ 编码从块开头开始的累积衰减，用于传播前一块的隐状态 $\mathbf{S}_{[i]}$ ； $\boxtimes_{[i+1]}$ 编码到块末尾的衰减，用于将信息累积到下一个隐状态 $\mathbf{S}_{[i+1]}$ 。

通俗解释

分块GLA中的三个关键衰减因子：

想象一个块覆盖位置 $[iC+1, (i+1)C]$ ：

$\boxtimes_{iC+j}$ （“从块头到当前的衰减”）：位置 j 查询前一块隐状态时，信息已经经过了 j 步衰减。类比：你在教室第 j 排，老师的声音传到你这里衰减了 j 排的距离。

$\boxtimes_{iC+j}$ （“从当前到块尾的衰减”）：位置 j 的信息要存入隐状态给下一块用，还需要经过 $(C-j)$ 步衰减。类比：你说的话要传到教室最后一排，还要衰减 $(C-j)$ 排。

γ_{i+1} （“整块的总衰减”）：前一块的隐状态 $\mathbf{S}_{[i]}$ 经过一整块的衰减。 $\gamma_{i+1} = \boxtimes_{(i+1)C} = \prod_{j=iC+1}^{(i+1)C} \alpha_j$ 。

5.3 硬件高效GLA

原文翻译

二级分块。 与普通线性注意力不同，GLA中的块内计算由于对数空间计算（公式 12）不能利用半精度矩阵乘法（从而不能用tensor core）。为了更好地利用tensor core，我们使用二级分块方案，将一个块进一步分成子块。注意力矩阵 $\mathbf{P}$ 以分块方式计算。具体地，**子块之间的交互**通过半精度矩阵乘法计算（对应图中的橙色块），而**子块内部**（粉色块）必须在全精度对数空间中计算。这种两级tiling策略大大减少了非半精度矩阵乘法的FLOPs。

内存高效的 $\mathbf{d}\alpha_t$ 计算。 之前的工作声称GLA类模型必须在HBM中实体化大小为 $L \times d \times d$ 的矩阵值隐状态才能计算所有梯度 $\mathbf{d}\alpha_t$ 。我们给出 $\mathbf{d} \log \alpha_t$ 的闭合形式公式：

$$\mathbf{d} \log b_t = \mathbf{q}_t \odot \mathbf{d} \mathbf{q}_t - \mathbf{k}_t \odot \mathbf{d} \mathbf{k}_t, \quad \mathbf{d} \log \alpha_t = \sum_{t \leq i \leq L} \mathbf{d} \log b_i,$$

这可以通过对公式 12求导容易地获得。 $\mathbf{d} \mathbf{q}_t$ 和 $\mathbf{d} \mathbf{k}_t$ 可以如反向传播算法计算。

关键概念/总结

二级分块是GLA的核心技术创新之一。它解决了一个看似矛盾的问题：门控的数据依赖性要求在对数空间计算（不能用tensor core），但效率又要求尽可能多用tensor core。

解决方案：把计算分成两种：

- 子块之间的交互：衰减因子可以聚合成子块级别的常数，然后用标准半精度矩阵乘法计算（✓ tensor core）
- 子块内部的计算：必须逐元素处理衰减因子，用全精度计算（× tensor core）

由于子块很小（如 16×16 ），第二种计算的比例很低，大部分计算都能利用tensor core。

$\mathbf{d}\alpha_t$ 闭合形式的意义：之前需要保存 $O(L \cdot d^2)$ 的隐状态来算梯度，现在只需要 $\mathbf{d}q_t$ 和 $\mathbf{d}k_t$ （已经在反向传播中计算了），内存从 $O(Ld^2)$ 降到 $O(Ld)$ 。

5.4 GLA Transformer 架构

原文翻译

我们将GLA层推广到多头情况。给定 H 个头，对每个头 $h \in [1, H]$ ：

$$\begin{aligned} \mathbf{S}_t^h &= \left(\left(\alpha_t^h \right)^\top \mathbf{1} \right) \odot \mathbf{S}_{t-1}^h + \mathbf{k}_t^{h\top} \mathbf{v}_t^h \in \mathbb{R}^{d'_k \times d'_v}, \\ \mathbf{o}_t^h &= \mathbf{q}_t^h \mathbf{S}_t^h \in \mathbb{R}^{1 \times d'_v}, \\ \mathbf{o}_t' &= \text{concat}(\text{LN}(\mathbf{o}_t^1), \dots, \text{LN}(\mathbf{o}_t^H)) \in \mathbb{R}^{1 \times d_v}, \\ \mathbf{r}_t &= \text{Swish}(\mathbf{x}_t \mathbf{W}_r + \mathbf{b}_r) \in \mathbb{R}^{1 \times d_v}, \\ \mathbf{y}_t &= (\mathbf{r}_t \odot \mathbf{o}_t') \mathbf{W}_O \in \mathbb{R}^{1 \times d}. \end{aligned}$$

每个头的输出后接LayerNorm，输出投影和输出门控操作在头输出的拼接上。然后通过交错多头GLA层和FFN构建Transformer式模型：

$$\begin{aligned} \mathbf{Y}^{(l)} &= \text{GLA}(\text{LN}(\mathbf{X}^{(l)})) + \mathbf{X}^{(l)} \\ \mathbf{X}^{(l+1)} &= \text{SwiGLU}(\text{LN}(\mathbf{Y}^{(l)})) + \mathbf{X}^{(l)}, \end{aligned}$$

其中SwiGLU FFN层为： $\text{SwiGLU}(\mathbf{Z}) = (\text{Swish}(\mathbf{Z}\mathbf{W}_1) \odot \mathbf{Z}\mathbf{W}_2)\mathbf{W}_3$ 。

参数分配。门控向量 α_t 使用低秩参数化： $\alpha_t = \sigma((\mathbf{x}_t \mathbf{W}_\alpha^1 \mathbf{W}_\alpha^2 + \mathbf{b}_\alpha))^{1/\tau}$ ，其中 $\mathbf{W}_\alpha^1 \in \mathbb{R}^{d \times 16}$ ， $\mathbf{W}_\alpha^2 \in \mathbb{R}^{16 \times d_k}$ ， $\tau = 16$ 是鼓励较慢遗忘率的温度项。设 $d_k = d/2$ ， $d_v = d_o$ 。最终一个GLA层大约需要 $4d^2$ 参数，与标准softmax注意力相同。

通俗解释

GLA Transformer的完整架构包含：

1. **多头GLA：**与多头注意力类似，将 d 维分成 H 个头，每个头独立计算带门控的线性注意力。关键区别是每个头输出后立即做LayerNorm——这对稳定训练很重要。
2. **输出门控：** $\mathbf{r}_t = \text{Swish}(\mathbf{x}_t \mathbf{W}_r)$ ，然后 $\mathbf{y}_t = (\mathbf{r}_t \odot \mathbf{o}_t') \mathbf{W}_O$ 。这个额外的门控（类似GLU）让模型可以进一步过滤输出信息。
3. **SwiGLU FFN：**这是LLaMA中使用的先进FFN，比标准ReLU FFN效果好。
4. **参数效率的关键设计：**

- α_t 通过 $d \rightarrow 16 \rightarrow d_k$ 的低秩映射生成，仅增加 $32d$ 参数

- 温度 $\tau = 16$ 使 $\sigma(\cdot)^{1/16}$ 的值更接近1，鼓励慢遗忘（保留更多历史）
- $d_k = d/2$ 减少key维度，节省参数但保持 $d_v = d$ 以维持表达力

6 实验研究 (Empirical Study)

6.1 实验设置

原文翻译

主要实验是语言建模任务，研究GLA能否在以下两方面表现出竞争力：(i) 使用现代架构配方的强Transformer基线；(ii) 最近的线性时间模型。使用SlimPajama数据集【Soboleva et al. 2023: SlimPajama, 627B token的清洗去重数据集】，使用Mistral分词器【Jiang et al. 2023: Mistral 7B】。使用100B子集。

基线：Transformer++【Touvron et al. 2023: LLaMA架构，带RoPE、SwiGLU、RMSNorm】、RetNet【Sun et al. 2023】和Mamba【Gu & Dao 2023】。

训练细节：所有模型从头训练，两个规模：340M和1.3B。使用AdamW【Loshchilov & Hutter 2018: AdamW优化器】，最大学习率 $3e-4$ 。340M模型在15B token上训练（batch size 0.5M token）；1.3B模型在100B token上训练（batch size 2M token）。余弦学习率调度。

6.2 主要结果

Table 2: GLA Transformer与Transformer++、RetNet和Mamba的比较。所有模型在相同的SlimPajama数据子集上使用Mistral分词器训练。340M/1.3B模型分别训练15B/100B个token。最后一列是所有使用（归一化）准确率指标的基准的平均值。

规模	模型	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	Avg. ↑
340M 15B tok	Transformer++	28.39	42.69	31.0	63.3	34.0	50.4	44.5	24.2	41.2
	RetNet	32.33	49.19	28.6	63.5	33.5	52.5	44.5	23.4	41.0
	Mamba	28.39	39.66	30.6	65.0	35.4	50.1	46.3	23.6	41.8
	GLA	28.65	43.35	30.3	64.8	34.5	51.4	45.1	22.7	41.5
1.3B 100B tok	Transformer++	16.85	13.44	48.9	70.8	49.6	53.6	56.0	26.5	50.9
	RetNet	18.64	17.27	43.3	70.0	47.3	52.5	54.8	25.6	48.9
	Mamba	17.06	13.89	46.2	72.2	40.1	54.1	59.0	28.2	50.0
	GLA	17.22	14.47	46.9	71.8	49.8	53.9	57.2	26.6	51.0

表 2 解读

这是论文最核心的结果表。关键发现：

1. GLA与Transformer++/Mamba总体持平。在1.3B规模上，GLA的平均准确率51.0%略高于Transformer++的50.9%和Mamba的50.0%。这表明线性注意力+数据依赖门控可以匹配标准Transformer的性能。

2. GLA远超RetNet。RetNet在1.3B规模上平均48.9%，显著落后。这证实了数据依赖门控相比数据无关的全局衰减的优势。

3. 各模型有不同的强项。Mamba在PIQA（72.2%）和ARC-e（59.0%）上最好；GLA在HellaSwag（49.8%）和Avg上最好；Transformer++在WikiText困惑度（16.85）上最好。

4. 规模提升后差距缩小。340M时模型差异较小，1.3B时GLA的优势更明显。

6.3 召回密集型任务

原文翻译

虽然亚二次方模型能达到与Transformer竞争的语言建模性能，但它们在召回密集型任务上落后于softmax注意力【Arora et al. 2024: 分析线性注意力的召回-吞吐量权衡】。

合成MQAR任务【Arora et al. 2023a (Zoology): 提出MQAR基准，测试模型的多查询关联召回能力】是归纳头任务的更具挑战性的多查询版本，模型必须多次召回查询token后面的token。标准二次方注意力在所有设置中都达到完美分数。具有矩阵值隐状态的模型（Mamba/RetNet/GLA）优于Hyena【Poli et al. 2023: Hyena层级，长卷积模型】/RWKV【Peng et al. 2023: RWKV，结合RNN和Transformer】，而GLA优于RetNet，确认了数据依赖门控的好处。

Table 3: 三个召回密集型任务的比较。越高越好。

规模	模型	FDA	SWDE	SQUAD
340M / 15B	Transformer++	21.4	42.2	22.1
	RetNet	2.9	13.3	27.6
	Mamba	2.1	12.4	23.0
	GLA	8.1	18.6	27.2
1.3B / 100B	Transformer++	27.4	66.6	31.5
	RetNet	14.3	42.8	34.7
	Mamba	6.2	41.4	35.2
	GLA	19.9	50.6	42.6

表 3 解读

这是一个非常重要的表格，揭示了不同架构在“信息检索”能力上的巨大差距：
Transformer++遥遥领先于信息提取任务（FDA、SWDE），因为softmax注意力可以“精确回看”任何历史位置。
GLA在亚二次方模型中最强：在FDA上19.9 vs Mamba的6.2，在SWDE上50.6 vs Mamba的41.4。这得益于GLA更大的隐状态（ $d_k \times d_v$ 矩阵vs Mamba受限于SRAM的小维度）和数据依赖的选择机制。
SQUAD上GLA最好（42.6），甚至超过Transformer++的31.5！这表明在阅读理解任务上，GLA的门控机制非常有效。
核心启示：隐状态大小（即“记忆容量”）是决定召回能力的关键因素。GLA的 $d_k \times d_v$ 矩阵比Mamba的SRAM受限隐状态大得多。

6.4 长序列训练与长度外推

原文翻译

线性注意力模型的一个优势是允许线性时间的高效长序列训练。我们考虑两种训练设置：(i) 直接在8K长度上下文上训练；(ii) 通过截断反向传播（TBPTT）在2K段上训练24K长度上下文。^a

GLA在大多数位置桶中外推效果优于Mamba/RetNet。Mamba在4K以后外推困难，而GLA/RetNet在SlimPajama测试集上可以泛化到18K。

^a将24K输入分成12段。前一段的最终状态作为当前段的初始状态。

通俗解释

长度外推是指在短序列（如2K）上训练后，模型能否在更长序列（如20K）上保持良好性能。

GLA的外推优势来源于：(1) 线性递推不依赖位置编码（不像RoPE受限于训练长度）；(2) 门控机制允许模型自适应地管理远距离信息的衰减。

TBPTT是一种经济的长序列训练方法：不需要真正在8K或24K上做完整反向传播，而是在2K段上计算梯度但在推理时传递隐状态。GLA在此设置下几乎与完整8K训练一样好。

6.5 消融实验

Table 4: 340M模型训练7B token的消融实验。评估最后200个训练步的平均困惑度。

模型变体	训练ppl.
GLA Transformer (4 heads)	14.77
无门控 (即线性注意力)	23.21
数据无关的标量衰减 (即RetNet)	16.55
数据依赖的标量门控	15.56
小头维度 (8 heads)	15.29
大头维度 (1 head)	14.61

表 4 解读

消融实验层层递进，揭示了每个设计选择的贡献：

无门控→数据无关衰减：23.21 → 16.55。衰减/遗忘机制贡献最大，无它线性注意力完全不能用。

数据无关→数据依赖标量门：16.55 → 15.56。让门控依赖于输入数据又提升了1分。

标量门→向量门 (GLA)：15.56 → 14.77。细粒度门控（每个维度不同的遗忘率）又提升了0.79。

头数的影响：1头（最大头维度）14.61最好但内存高；8头（最小头维度）15.29略差；4头是性能-效率的最佳折中。

6.6 训练效率

原文翻译

图 ??展示了不同1.3B模型在单个H100 GPU上，吞吐量和内存使用随序列长度和batch size的变化。^aGLA采用带重计算的实体化版FlashLinearAttention。所有模型的线性空间复杂度相当，GPU总占用差异最小。在训练吞吐量方面，Mamba落后于Transformer++和GLA，GLA在训练长度超过4096时显示更大优势。

^a对Mamba使用官方实现，对Transformer++和GLA使用融合版SwiGLU，对Transformer++使用FlashAttention-2。

关键概念/总结

训练效率总结：

- 吞吐量：GLA > Transformer++ > Mamba（序列长度4K以上差距更大）
- 内存：三者相当（约33-37GB对1.3B模型）
- GLA的优势来自：(1) 充分利用tensor core；(2) 兼容tensor parallelism（多头结构）

- Mamba的劣势来自：(1) 不能用tensor core；(2) 不是多头结构，不利于tensor parallelism

7 相关工作 (Related Work)

原文翻译

传统RNN由于隐状态之间的非线性依赖和基于矩阵乘法的顺序隐状态更新而难以扩展。线性RNN/状态空间模型(SSM)/Transformer消除了非线性依赖，使训练可以沿时间维度并行化【Martin & Cundy 2018: 首次提出并行化线性递推】【Gu et al. 2022 (S4): 开创性的结构化状态空间模型】【Smith et al. 2023 (S5): 简化的状态空间层】。数据依赖的衰减率一直被认为对RNN重要【Gers et al. 2000: LSTM遗忘门的原始论文】【van der Westhuizen & Lasenby 2018】。Martin & Cundy建议遗忘门值应仅依赖于当前输入以实现并行训练【Martin & Cundy 2018】。线性Transformer通过外积参数化扩展RNN的隐藏维度。线性SSM则通过单输入单输出(SISO)策略扩展。不带数据依赖参数时，可以通过FFT高效训练。有数据依赖参数时，FFT不可用，因此Mamba使用并行扫描算法的自定义CUDA核。为将所有隐状态装入SRAM，扩展率最高只能到16。而我们的硬件感知训练算法提供了一种替代方案，隐藏维度可以扩展到更大范围。

通俗解释

相关工作梳理了三条发展线索：

线索1——线性RNN/SSM的崛起： S4 → S5 → Mamba，核心是用线性递推替代非线性RNN，实现并行训练。

线索2——门控/衰减的演进： 固定衰减 (RetNet/ALiBi) → 数据依赖标量门 (Mamba-2/mLSTM) → 数据依赖向量门 (GLA/HGRN-2/RWKV-6) → 完全低秩矩阵门 (DFW)。

线索3——隐状态维度扩展： 1D向量 (传统RNN) → 2D矩阵 (线性注意力的 $d \times d$ 外积) → 多项式核 ($O(d^p)$ 维)。隐状态越大，记忆容量越强，但训练越难。

8 结论 (Conclusion)

原文翻译

我们提出了一种高效算法，用于训练具有数据依赖门控机制的线性注意力Transformer。我们的算法使得可以在FLOPs和并行性之间取得平衡，同时仍然允许使用半精度矩阵乘法来利用现代GPU上的tensor core单元。语言建模实验表明，门控线性注意力Transformer可以与强基线相比表现出色。

影响声明

本文旨在提高（门控）线性注意力模型的训练效率。这类模型的效率优势可能有助于民主化语言模型的访问。另一方面，此类新架构是否会影响语言模型的已知问题（如偏见和有害输出）仍是一个未探索的研究问题。

致谢

本工作得到MIT-IBM Watson AI Lab的支持。感谢Yutao Sun、Zhen Qin、Li Dong、Xinyu Yang、Jiacheng You、Huanqi Cao、Yu Zhang和Shida Wang的深入讨论。感谢Yu Zhang、Fares Obeid、Daniel Goldstein和Liliang Ren的校对。特别感谢Yu Zhang对FlashLinearAttention库的贡献。

9 附录A: 扩展相关工作

原文翻译

特征映射 ϕ 。各种核和特征映射已被探索： $1 + \text{elu}$ 、ReLU、 $\exp(\cdot)$ 、随机特征、可学习MLP和恒等映射。本文使用恒等映射。

注意力稀疏性。线性注意力存在“注意力稀释”问题，注意力分布过于均匀。解决方案包括：添加局部注意力层、缩放指数映射 $\phi(\mathbf{x}) = \exp(t \cdot \mathbf{x})$ ($t \geq 2$)。

记忆容量。线性注意力具有有界记忆大小，而softmax注意力享有无界记忆。增加记忆大小的方法：直接增加 d_{key} （但参数难控制）、高阶多项式核、确定性无参数投影（DPFP）、参数化外积扩展。

线性注意力的衰减/门控。标量门控在分块线性注意力中易于实现，矩阵值门控的训练效率更具挑战。之前的工作需要在HBM中实体化所有步的隐状态且不能利用tensor core。我们的算法减少或消除了实体化并启用了tensor core。

I/O感知的分块线性注意力。并发工作LightningAttention-2与我们的非实体化版本非常相似。我们额外提出了实体化版本，利用序列级并行性实现更高训练吞吐量。

10 附录B: 分块（门控）线性注意力的详细算法

10.1 FlashLinearAttention 反向传播

算法 2 解读

反向传播的关键是计算 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ （损失对 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 的梯度）。

实体化版本也分两个阶段：

- **阶段1：**逆序扫描计算 $\mathbf{dS}_{[n]}$ （隐状态的梯度），这是串行的
- **阶段2：**对所有块并行计算 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$

每个块的梯度分两部分——来自块间（通过 $\mathbf{S}_{[n]}$ 和 $\mathbf{dS}_{[n]}$ ）和来自块内（通过掩码矩阵 $\mathbf{M}$ ），与前向传播对称。

非实体化版本需要两次扫描：正序扫描重计算 $\mathbf{S}$ 以计算 $\mathbf{dQ}$ ，逆序扫描计算 $\mathbf{dS}$ 和 $\mathbf{dK}, \mathbf{dV}$ 。

10.2 GLA实体化版本的前向传播

算法 3 解读

相比基础FlashLinearAttention，GLA版本的关键区别：

1. **隐状态更新带衰减：** $\mathbf{S} = (\gamma_{[n]}^T \mathbf{1}) \odot \mathbf{S} + \tilde{\mathbf{K}}_{[n]}^T \mathbf{V}_{[n]}$ ，其中 $\gamma_{[n]}$ 是整个块的累积衰减， $\tilde{\mathbf{K}}_{[n]} = \mathbf{K}_{[n]} \odot \boxtimes_{[n]}$ 是经过衰减调制的key。
2. **块内计算中Q和K都要调制：** $\tilde{\mathbf{Q}}$ 和 $\tilde{\mathbf{K}}$ 分别用 $\boxtimes$ 和 $1/\boxtimes$ 调制，这编码了块内各位置的相对衰减。
3. **块间传播也带衰减：** $\mathbf{O}^{\text{inter}} = \tilde{\mathbf{Q}} \cdot \mathbf{S}_{[n]}$ 中的 $\tilde{\mathbf{Q}}$ 已经包含了衰减信息。这些衰减因子使得GLA能够区别对待新旧信息——越旧的信息衰减越多，新信息影响更

输入: $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{L \times d}$, 块大小 C , $\text{materialize} \in \{\text{True}, \text{False}\}$, $\mathbf{S} \in \mathbb{R}^{\frac{L}{C} \times d \times d}$

初始化 $\mathbf{dS} = \mathbf{0} \in \mathbb{R}^{d \times d}$ 于 SRAM

在片上构造因果掩码 $\mathbf{M} \in \mathbb{R}^{C \times C}$

if materialize then

for $n \leftarrow N, 1$ do

将 $\mathbf{dS}$ 存储到 HBM 为 $\mathbf{dS}_{[n]}$

从 HBM 加载 $\mathbf{Q}_{[n]}, \mathbf{dO}_{[n]} \in \mathbb{R}^{C \times d}$ 到 SRAM

在片上计算 $\mathbf{dS} = \mathbf{dS} + \mathbf{Q}_{[n]}^\top \mathbf{dO}_{[n]}$

end for

parfor $n \leftarrow 1, N$ do

从 HBM 加载 $\mathbf{Q}_{[n]}, \mathbf{K}_{[n]}, \mathbf{V}_{[n]}, \mathbf{dO}_{[n]}, \mathbf{S}_{[n]}, \mathbf{dS}_{[n]}$ 到 SRAM

$\mathbf{dQ} = \mathbf{dO}_{[n]} \mathbf{S}_{[n]}^\top + (\mathbf{dO}_{[n]} \mathbf{V}_{[n]}^\top \odot \mathbf{M}) \mathbf{K}_{[n]}$

$\mathbf{dK} = \mathbf{V}_{[n]} \mathbf{dS}_{[n]}^\top + (\mathbf{V}_{[n]} \mathbf{dO}_{[n]}^\top \odot \mathbf{M}^\top) \mathbf{Q}_{[n]}$

$\mathbf{dV} = \mathbf{K}_{[n]} \mathbf{dS}_{[n]} + (\mathbf{Q}_{[n]} \mathbf{K}_{[n]}^\top \odot \mathbf{M})^\top \mathbf{dO}_{[n]}$

将 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ 写入 HBM

end parfor

else

初始化 $\mathbf{S} = \mathbf{0} \in \mathbb{R}^{d \times d}$ 于 SRAM

for $n \leftarrow 1, N$ do

加载 $\mathbf{K}_{[n]}, \mathbf{V}_{[n]}, \mathbf{dO}_{[n]}$ 到 SRAM

$\mathbf{dQ} = \mathbf{dO}_{[n]} \mathbf{S}^\top + (\mathbf{dO}_{[n]} \mathbf{V}_{[n]}^\top \odot \mathbf{M}) \mathbf{K}_{[n]}$

$\mathbf{S} = \mathbf{S} + \mathbf{K}_{[n]}^\top \mathbf{V}_{[n]}$

end for

for $n \leftarrow N, 1$ do

加载 $\mathbf{Q}_{[n]}, \mathbf{K}_{[n]}, \mathbf{V}_{[n]}, \mathbf{dO}_{[n]}$ 到 SRAM

$\mathbf{dS} = \mathbf{dS} + \mathbf{Q}_{[n]}^\top \mathbf{dO}_{[n]}$

$\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ 计算同实体化版本

写入 HBM

end for

end if

return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$

▷ 实体化版本

▷ 逆序

▷ 非实体化版本

▷ 隐状态重计算

▷ 逆序

输入: $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{L \times d_k}, \mathbf{V} \in \mathbb{R}^{L \times d_v}, \mathbf{G} = [\alpha_1 \dots \alpha_L] \in \mathbb{R}^{L \times d_k}$, 块大小 C

将 $\mathbf{Q}, \mathbf{K}, \mathbf{G}$ 分成 N 块, $\mathbf{V}$ 分成 N 块

初始化 $\mathbf{S} = \mathbf{0} \in \mathbb{R}^{d_k \times d_v}$ 于 SRAM

for $n \leftarrow 1, N$ do

将 $\mathbf{S}$ 写入 HBM 为 $\mathbf{S}_{[n]}$

加载 $\mathbf{K}_{[n]}, \mathbf{G}_{[n]}, \mathbf{V}_{[n]}$ 到 SRAM

在片上计算 $\gamma_{[n]}, \boxtimes_{[n]}$ 及 $\tilde{\mathbf{K}}_{[n]} = \mathbf{K}_{[n]} \odot \boxtimes_{[n]}$

在片上计算 $\mathbf{S} = (\gamma_{[n]}^\top \mathbf{1}) \odot \mathbf{S} + \tilde{\mathbf{K}}_{[n]}^\top \mathbf{V}_{[n]}$

end for

parfor $n \leftarrow 1, N$ do

加载 $\mathbf{Q}_{[n]}, \mathbf{K}_{[n]}, \mathbf{G}_{[n]}, \mathbf{V}_{[n]}, \mathbf{S}_{[n]}$ 到 SRAM

在片上计算 $\boxtimes_{[n]}, \boxtimes_{[n]}$

计算 $\tilde{\mathbf{Q}}_{[n]} = \mathbf{Q}_{[n]} \odot \boxtimes_{[n]}, \tilde{\mathbf{K}}_{[n]} = \mathbf{K}_{[n]} \odot \boxtimes_{[n]}, \bar{\mathbf{K}}_{[n]} = \mathbf{K}_{[n]} / \boxtimes_{[n]}$

$\mathbf{O}_{[n]}^{\text{inter}} = \tilde{\mathbf{Q}}_{[n]} \mathbf{S}_{[n]}$

$\mathbf{P} = (\tilde{\mathbf{Q}}_{[n]} \bar{\mathbf{K}}_{[n]}^\top) \odot \mathbf{M}$

$\mathbf{O}_{[n]}^{\text{intra}} = \mathbf{P} \mathbf{V}_{[n]}$

$\mathbf{O}_{[n]} = \mathbf{O}_{[n]}^{\text{inter}} + \mathbf{O}_{[n]}^{\text{intra}}$

将 $\mathbf{O}_{[n]}$ 存储到 HBM

end parfor

return $\mathbf{O}, \mathbf{S}$